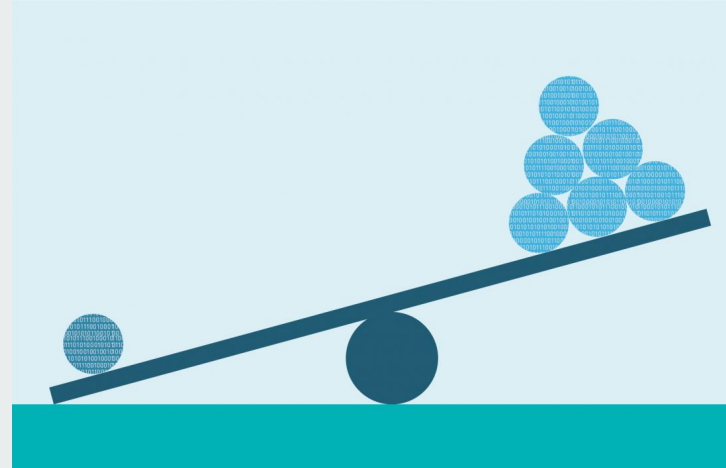




Tractar amb dades desequilibrades





Desequilibri del target (imbalance)

- Els models de ML tendeixen a afavorir la classe target majoritària.
- Conseqüències:
 - Rendiment reduït del model, especialment per a la classe minoritària.
 - Augment de la probabilitat d'overfitting a la classe majoritària.



Desequilibri del target (imbalance)

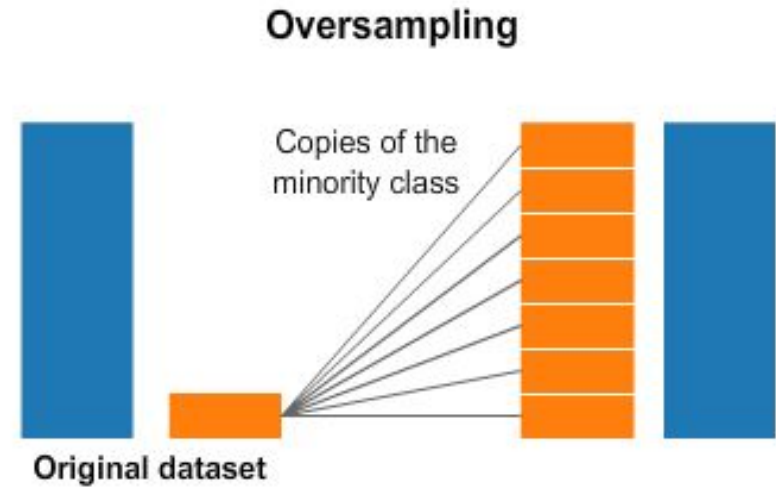
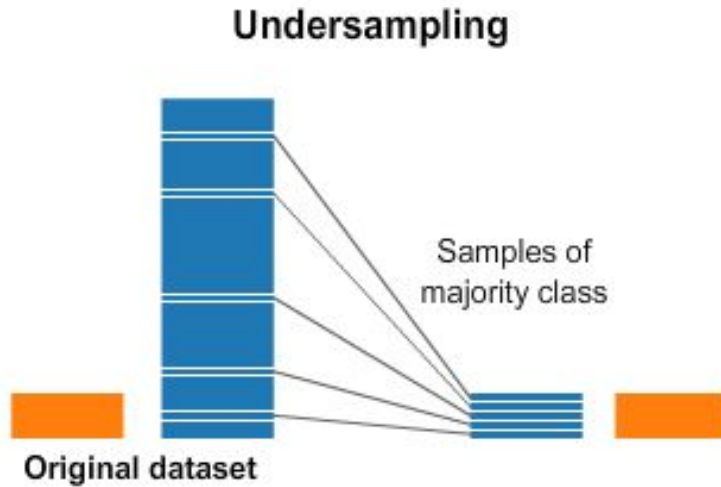
- Els models de ML tendeixen a afavorir la classe target majoritària.
- Conseqüències:
 - Rendiment reduït del model, especialment per a la classe minoritària.
 - Augmentada probabilitat d'overfitting a la classe majoritària.
- Tècniques comunes:
 - Donar més pes a la classe minoritària.
 - Equilibrar el nombre de mostres de cada classe.



Pes de la classe

- A *sklearn*, la majoria d'algorismes de classificació tenen un hiperparàmetre *class_weight*.
- Modifica la funció de loss per canviar el pes de les classes al construir el model.
- Dues opcions:
 - **"balanced"** ajusta automàticament els pesos de manera inversament proporcional a les freqüències de les classes.
 - Com més rara és la classe, més pes obté.
 - Diccionari indicant els pesos de les classes. Per exemple: {0: 1, 1: 3}
 - Aquí, la classe 1 es considera 3 vegades més important que la classe 0.
 - És a dir, 3 elements de la classe 1 seran equivalents a 1 element de la classe 0.

Undersampling i Oversampling



Undersampling i Oversampling

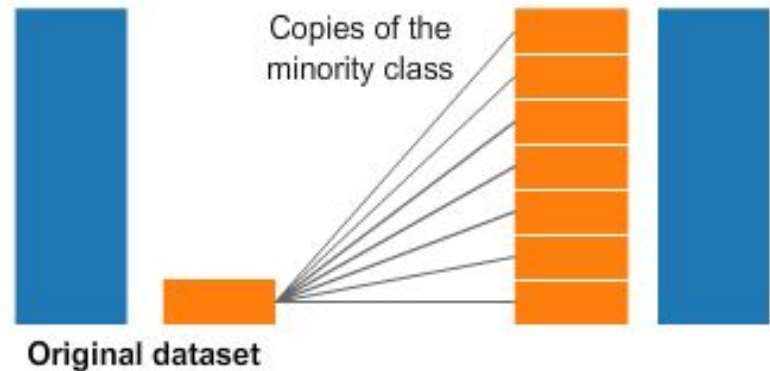
Només s'aplica a les dades d'entrenament!

Les dades de validació/test han de mantenir la proporció original

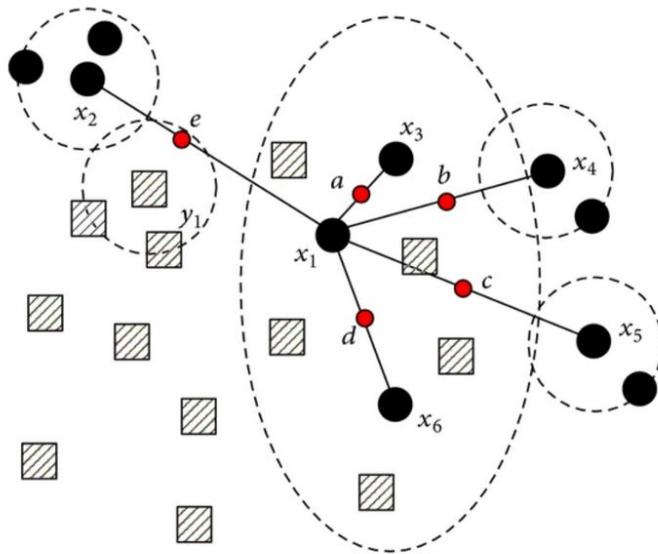
Undersampling



Oversampling

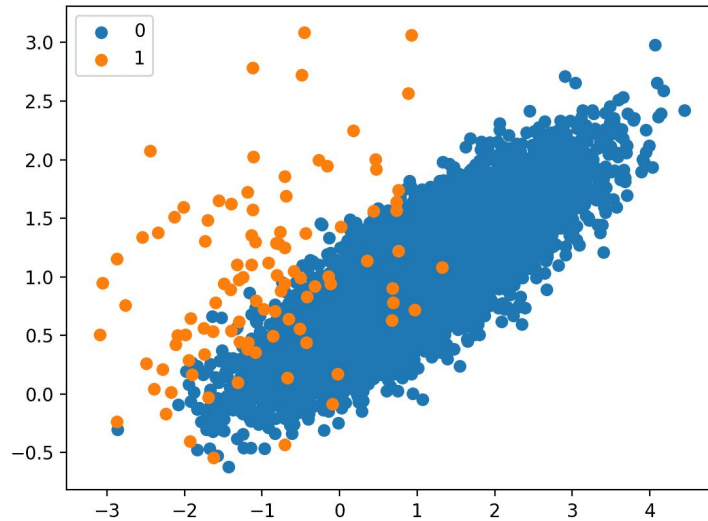


Oversampling: SMOTE

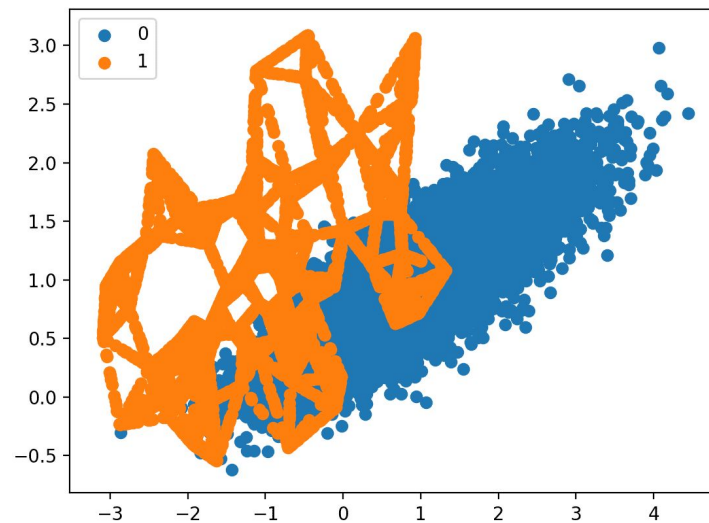
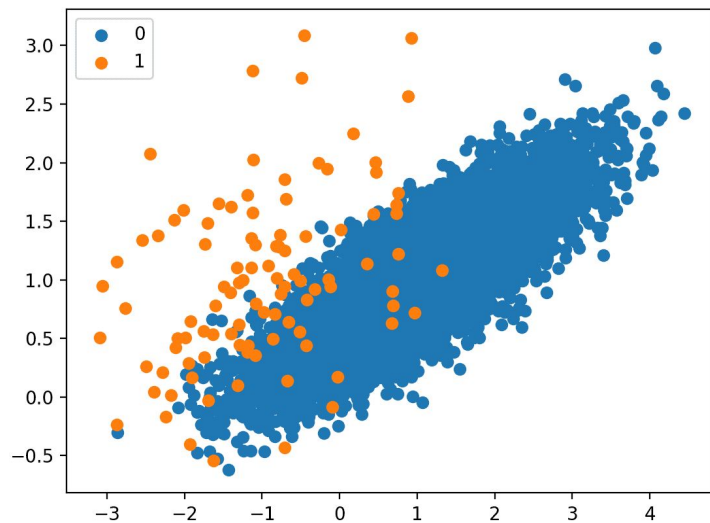


- Majority class samples
- Minority class samples
- Synthetic samples

SMOTE



SMOTE





Imblearn

- La llibreria `imblearn` té funcionalitats per equilibrar les classes.
- Per defecte, el *Pipeline* de `sklearn` no permet modificar l'objectiu `y`.
- Per tant, hem d'utilitzar el *Pipeline* de `imblearn`.
- Quan utilitzem l'objecte *Pipeline*, les transformacions de dades corresponents s'aplicaran només al conjunt de train i **no** al de validació/test, tal i com volem.

Imblearn: Exemple de Random Undersampling



```
from sklearn.linear_model import LogisticRegression
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.under_sampling import RandomUnderSampler

lr_pipe = ImbPipeline([
    ('RUS', RandomUnderSampler()),
    ('LR', LogisticRegression())
])
```

Imblearn: Exemple de SMOTE



```
from sklearn.linear_model import LogisticRegression
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.over_sampling import SMOTE

lr_pipe = ImbPipeline([
    ('SMOTE', SMOTE(random_state=42)),
    ('LR', LogisticRegression())
])
```



Combinació d'undersampling i oversampling

- Exemple:
 - Undersampling d'un percentatge de la classe majoritària.
 - Després oversampling del percentatge restant de la classe minoritària.

Balanced Random Forest

- Undersampling aleatori per a cada arbre.
- Hi ha una implementació de *BalancedRandomForestClassifier* a *imblearn*.

